

CCP6224 Object Oriented Analysis and Design

Lab 7 – Design Patterns

Lab outcomes

By the end of today's lab, you should be able to

- differentiate behavioural and creational design patterns and apply them to particular scenarios (in addition to structural)

Design patterns

1. Map the reasons of when to implement with the correct matching design pattern

<i>DP</i>	<i>Reasons to implement</i>
Singleton •	• used to access elements in a collection of objects in a sequential manner
Prototype •	• to add functionality to an instantiated object during runtime instead of statically during compilation
Builder •	• to create complex objects from individual parts in certain order/algorithm
Decorator •	• to limit an object to a single instance
Iterator •	• adapts existing legacy code
Observer •	• to create duplicate objects without affecting application performance
Strategy •	• used when the operating behaviour or algorithm of a class needs to be changed during runtime
	• represent multiple related objects (usually a hierarchy) with a generalized object such that each object and composition of objects are treated uniformly (i.e. same interface)
	• used when there is a one-to-many relationship between objects such that when one (source) object is modified the dependent (observing) objects are notified

2. In an implementation of a board game, there is one game board that will keep track of many pieces of information. There should only be one game board. All the other objects in the system, when referring to a game board, should end up with the same game board.

Design pattern:

Reasoning:

3. You are working on the Romanas Project, and in that project, you have a Zogny object with an undo operation such a Zogny process could simply maintain a list of all previous operations that it has performed and thus would be able to restore a previous step it has performed. This would cause the Zogny object to become larger, more complex, and heavyweight, as the Zogny object would have to provide additional undo functionality and should maintain a list of all previous operations.

This functionality needs to be moved out of the Zogny class, so that an external undo manager class can collect the internal state of the Zogny process thus far and save it. However providing the explicit access to every state variable of the Zogny to the restore manager would be impractical and would violate the encapsulation principle.

Design pattern:

Reasoning:

4. In a manufacturing system, the system needs to, when started up, survey all the machines in the system and get their configurations. Then, different objects will be customized for different processes, and they will use these configurations. Depending on the processes, these objects may get further customized. The survey process is relatively time-consuming (on the order of seconds) while the process-minding objects may get spawned off fairly regularly as new jobs appear in the manufacturing queue.

Design pattern:

Reasoning:

5. The Thagomizer is a sophisticated piece of equipment, and it has various sensors to track its states. You are writing module which monitors those sensors. Many other modules might need to know when various sensors is triggered, but you do not know at the time of designing your module what those other modules are.

Design pattern:
Reasoning:

6. Project XIF has a lot of customizable processes. In order to make it easier for end users to customize those processes, it has been decided that XIF should use the scripting language FIX.

Design pattern:
Reasoning:

7. You work for a division that produces a general-purpose controller for machinery that your company manufactures. Because of the large number of possible machines it will control, when instantiating an instance of this controller, there are many possible parameters that could be given. For any particular instance, not all the parameters might be relevant, so you need to accommodate many possible combinations of parameters in your object creation.

Design pattern:
Reasoning: